COMPTIA SECURITY+ SY0-701 | DOMAIN 2 · THREATS, VULNERABILITIES, AND MITIGATIONS (22%)

Security+ 2.4 · Replay Attacks

Mestre CyberSec · **Lesson 2.4 — Replay Attacks** (5:45) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Executive Lesson Summary: Replay: capturar o handshake/token e REENVIAR pra se autenticar de novo. Pass-the-hash: usar o HASH da senha como autenticador direto NTLM (Mimikatz + PsExec) — movimento lateral. Session hijacking: roubar cookie de sessão ativa (via XSS ou on-path) e se passar pelo usuário. Defe

💡 Key Exam Highlights & Critical Concepts

- Replay: reenvia o 'handshake' capturado — defesa: nonce/timestamp
- Pass-the-hash: usa O HASH (não a senha) — NTLM é vulnerável, Kerberos menos
- Session hijacking: rouba cookie — HttpOnly + HTTPS mitiga

🎯 Exam Tips & Typical CompTIA Scenarios

- 'reuses captured authentication data' = replay attack
- 'uses an NTLM hash to authenticate' = pass-the-hash

🚨 Beyond the Video: Official Exam Concepts, Protocols & Traps (Section 2.4 — Replay Attacks)

Why is this advanced coverage required for SY0-701?: Video lectures provide foundational concepts, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

• Replay Attack

An attack that captures valid authentication data, such as a handshake or token, and retransmits it to authenticate as the original user. It does not break encryption; it reuses captured credentials.

⚠️ ***Exam Trap:*** *Gatilho: 'reuses captured authentication data' = replay attack. Defesa principal: nonce e timestamp para tornar o dado único e não reutilizável.*

• Pass-the-Hash (PtH)

An attack that uses a captured NTLM password hash as an authenticator directly, without cracking it to plaintext. Commonly uses Mimikatz to dump hashes and PsExec for lateral movement.

⚠️ ***Exam Trap:*** *Gatilho: 'uses an NTLM hash to authenticate' = pass-the-hash. NTLM é vulnerável; Kerberos reduz esse risco.*

• Session Hijacking

An attack that steals an active session cookie or token, often via XSS or on-path interception, and uses it to impersonate the authenticated user.

⚠️ ***Exam Trap:*** *Gatilho: 'steals session cookie' = session hijacking. Mitigações: HttpOnly e HTTPS impedem acesso via script e protegem o cookie em trânsito.*

• Nonce

A random or unique value used once in an authentication exchange to ensure freshness and prevent replay of captured messages.

⚠️ ***Exam Trap:*** *Defesa contra replay: nonce torna cada handshake único. Se a prova citar 'number used once', é nonce.*

• Timestamp

A time value included in authentication data to verify freshness and reject messages that are too old to be valid.

⚠️ ***Exam Trap:*** *Defesa contra replay: se o timestamp estiver fora da janela aceitável, a autenticação é negada. Distinga de nonce: tempo vs. valor único.*

• **NTLM**

A challenge-response authentication protocol in Windows that can be abused in pass-the-hash because possession of the password hash is enough to authenticate.

⚠️ ***Exam Trap:*** *Gatilho: NTLM + hash = pass-the-hash. Kerberos é preferível e menos suscetível a esse tipo de ataque.*

• **Kerberos**

A ticket-based authentication protocol that is less vulnerable to pass-the-hash than NTLM because it does not rely on sending password hashes for authentication.

⚠️ ***Exam Trap:*** *Distinga: NTLM é associado a pass-the-hash; Kerberos usa tickets e é a alternativa mais segura em domínios Windows.*

• **HttpOnly**

A cookie attribute that prevents client-side scripts, such as JavaScript, from accessing the cookie, reducing session theft via XSS.

⚠️ ***Exam Trap:*** *Mitigação de session hijacking: HttpOnly protege contra XSS; HTTPS protege o cookie em trânsito. Se a questão citar 'script access to cookie', responda HttpOnly.*

🏛️ Conceptual Deep Dive, Real-World Analogies & GRC Analysis

📌 Server-Side Request Forgery (SSRF)

🏠 **Real-World Analogy:** SSRF is like standing outside a high-security embassy and handing a task memo to the official internal courier: 'Please step into the classified vault on floor 8, photograph the ambassador's ledger, and bring the printout to me at the curbside desk'. Because the courier holds clearance to navigate interior corridors inaccessible to the general public, they faithfully retrieve the classified internal file and hand it directly to the outsider.

🔍 **Root Cause & Background:** Web applications frequently fetch remote resources (importing images or fetching webhooks). If the server accepts unvalidated user-supplied destination URLs, attackers supply internal IP targets (loopback `127.0.0.1`, RFC 1918 private subnets) or cloud instance metadata endpoints (`169.254.169.254`), exfiltrating ephemeral IAM credentials.

🛡️ **GRC Auditor Perspective:** Enforce strict destination allowlists on server outbound calls; disable alternative URI schemes (e.g., `file://`); block loopback and private IP resolution; and mandate cloud Instance Metadata Service Version 2 (IMDSv2) requiring session tokens.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker manipulates a web app into issuing unauthorized requests targeting internal private addresses or cloud metadata (169.254.169.254)' -> Server-Side Request Forgery (SSRF).

📌 Cross-Site Request Forgery (CSRF)

🏠 **Real-World Analogy:** You have an active, authenticated tab open with your online bank. In another browser tab, you visit an untrusted forum or deceptive link. That third-party site stealthily fires an HTTP POST request to your bank: 'Transfer $1,000 to adversary account 999'. Because your web browser automatically includes your valid session cookies with any request addressed to that bank, the banking server assumes you intentionally triggered the action and executes the wire.

🔍 **Root Cause & Background:** Browsers automatically attach ambient session cookies whenever dispatching requests to target origins. Without anti-forgery validation tokens tied to specific form renders, servers cannot discern whether an action was deliberately initiated by the user or forged across origins.

🛡️ **GRC Auditor Perspective:** Mandate cryptographically random Anti-CSRF Synchronizer Tokens embedded within state-changing forms; enforce the `SameSite=Strict` or `SameSite=Lax` attribute on session cookies; and require step-up reauthentication for critical actions.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker leverages an authenticated victim session to force unauthorized actions on a trusted website' -> Cross-Site Request Forgery (CSRF). Primary defenses: Anti-CSRF Tokens and SameSite cookies.

📌 Broken Object Level Authorization (BOLA) & API Security

🏠 **Real-World Analogy:** BOLA (historically IDOR) is like renting locker 101 at a gym. After your workout, you walk past locker 102, jiggle the handle, and the door swings open, exposing another member's valuables. The gym verified you are a paying member permitted inside the facility (valid authentication), but completely neglected to verify whether you hold the legal authority to open locker 102 specifically.

🔍 **Root Cause & Background:** In modern REST APIs, clients reference data objects via request parameters (e.g., `/api/records/402`). While the API validates that the client holds an active auth token (Authentication), it fails to check if that user is legitimately authorized to access object 402 (Authorization). BOLA ranks as the #1 threat in the OWASP API Security Top 10.

🛡️ **GRC Auditor Perspective:** Audit API routes to ensure object-level ownership checks (ABAC) run prior to database record retrieval; substitute sequential integer IDs with cryptographically random UUIDv4s; and deploy API Gateways enforcing centralized policy checks.

⚠️ **How CompTIA Tests This:** Trigger: 'authenticated user changes an ID parameter in an API call from 2001 to 2002 and views another customer's confidential statement' -> Broken Object Level Authorization (BOLA) / IDOR.

📌 HTTP Parameter Pollution (HPP)

🏠 **Real-World Analogy:** It is like completing an invoice voucher that features a 'Payee:' line, and writing the payee twice: on the top line you write the vetted vendor name, but on the bottom line you write the fraudster's name. The front-desk inspector (the WAF) reviews only the first line, approving the voucher; the disbursement cashier (the backend database) reads only the last line, sending the funds to the fraudster. Discrepancies between how components parse identical keys create security bypasses.

🔍 **Root Cause & Background:** Web application frameworks handle duplicate HTTP parameters inconsistently (some parse only the first value, others only the last, while others concatenate them). Adversaries exploit these parsing mismatches between perimeter WAFs and backend servers (`?role=user&role=admin`) to bypass validation filters.

🛡️ **GRC Auditor Perspective:** Standardize parameter parsing logic across front-end reverse proxies and backend servers; reject requests containing duplicate parameter occurrences; and enforce strict OpenAPI input schemas.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker injects duplicate parameters with identical names to bypass security filters caused by parsing differences between the WAF and backend' -> HTTP Parameter Pollution (HPP).

⚡ QUICK-REFERENCE EXAM CHEAT SHEET

Typical Exam Scenario / Practical Situation	Technical Concept & Target Answer
Attacker coerces a web application into making unauthorized requests to internal IP addresses or the cloud metadata service	**Server-Side Request Forgery (SSRF)**
Malicious site tricks an authenticated browser into submitting state-changing transactions against a trusted application	**Cross-Site Request Forgery (CSRF)**
Authenticated client alters an ID parameter in an API endpoint request to access unauthorized records belonging to others	**Broken Object Level Authorization (BOLA) / IDOR**
Attacker supplies duplicate parameter names in an HTTP query to bypass WAF validation filters due to parsing differences	**HTTP Parameter Pollution (HPP)**
Defensive countermeasure attaching unique unpredictable tokens to forms ensuring only deliberate user actions execute	**Anti-CSRF Token (Synchronizer Token)**
AWS metadata service version requiring session token generation via HTTP PUT requests, neutralizing SSRF attempts	**IMDSv2 (Instance Metadata Service Version 2)**

✅ Authentic Practice Questions (10 Real Exam Scenarios)

Scenario-based exam questions. Complete explanations and distractor analysis are provided in the answer key below.

QUESTION 1 · EXAM SCENARIO

A financial web application accepts signed API requests for account transfers. A penetration tester records a legitimate transfer request and successfully submits the exact same request again later without changing it. The server currently verifies only the digital signature. Which control should be added to prevent this attack?

A) Hash the request body with SHA-256 before signing it.

B) Include an unpredictable one-time value in each request and reject any reused value.

C) Require TLS 1.3 for all API connections.

D) Force users to rotate their passwords every 90 days.

QUESTION 2 · EXAM SCENARIO

During an incident, analysts find that an attacker obtained credential material from one workstation and used it to authenticate to several file servers. The attacker never cracked the password and never knew the plaintext value. Which attack technique does this describe?

A) Session hijacking

B) Kerberoasting

C) Pass-the-hash

D) Replay attack

QUESTION 3 · EXAM SCENARIO

A web application stores session identifiers in cookies. A security tester demonstrates that injected JavaScript can read the session cookie and send it to an external server. Which cookie attribute most directly prevents this specific script-based theft?

A) Secure

B) SameSite=Strict

C) HttpOnly

D) Domain

QUESTION 4 · EXAM SCENARIO

Employees use a public Wi-Fi network. An attacker on the same network captures an authenticated session cookie sent over HTTP and uses it to access the employee's account. Which mitigation best addresses this risk?

A) Set session cookies with the Secure flag and enforce HTTPS with HSTS.

B) Store session identifiers in browser local storage and validate them only on the client side.

C) Reduce session timeout values to five minutes.

D) Enable HTTP compression and caching for the application.

QUESTION 5 · EXAM SCENARIO

A Windows domain uses Kerberos authentication. Security architects want to ensure that an attacker who captures a valid authentication exchange cannot simply retransmit it later. Which mechanism does Kerberos rely on to limit this type of reuse?

A) An NTLM hash cache

B) A timestamped authenticator

C) A session cookie with SameSite

D) A permanent service ticket

QUESTION 6 · EXAM SCENARIO

An organization wants to reduce the risk that an attacker who compromises one Windows workstation can use cached credential material to authenticate to other systems as an administrator. Which control is most effective?

A) Enable browser HttpOnly and Secure cookie flags.

B) Deploy Credential Guard and restrict privileged accounts to dedicated administration systems.

C) Implement CAPTCHA on the VPN login page.

D) Require AES encryption for all Kerberos service accounts.

QUESTION 7 · EXAM SCENARIO

A custom authentication protocol sends a password-derived response to a server. An attacker captures the response and authenticates successfully by sending the exact same response again. Which design change best prevents this?

A) Increase the minimum password length.

B) Encode the response with Base64 before transmission.

C) Use a server-generated random challenge in each authentication attempt and include it in the response.

D) Store every password-derived response in a database for auditing.

QUESTION 8 · EXAM SCENARIO

A web application assigns a session ID when a user first visits the site and keeps the same ID after the user logs in. An attacker can set a known session ID in the victim's browser and then use it after the victim authenticates. Which remediation best addresses this vulnerability?

A) Mark the session cookie as HttpOnly.

B) Add a nonce to each login request.

C) Regenerate the session ID immediately after successful authentication.

D) Enable NTLM authentication for the web server.

QUESTION 9 · EXAM SCENARIO

SIEM alerts show multiple successful NTLM authentications from a single user account to many servers. The user's workstation has no record of interactive password entry, and the source process is not a browser. Which activity is most consistent with these indicators?

A) Cross-site scripting stealing cookies

B) Pass-the-hash using a stolen credential hash

C) Kerberos ticket replay

D) Session fixation

QUESTION 10 · EXAM SCENARIO

A multi-factor authentication system accepts a six-digit one-time passcode for five minutes. An attacker intercepts a valid passcode and uses it within the allowed window. Which improvement best prevents reuse of the same passcode?

A) Increase the passcode length to eight digits.

B) Switch from an authenticator app to SMS delivery.

C) Track each passcode as used and bind it to the specific authentication session or challenge.

D) Store issued passcodes in browser local storage for faster validation.

ANSWER KEY & COMPREHENSIVE DISTRACTOR EXPLANATIONS

Question 1: Option B — Include an unpredictable one-time value in each request and reject any reused value.
A digital signature proves integrity and origin but does not stop an attacker from replaying a captured valid request. A nonce is a unique, unpredictable value used once; the server tracks used nonces and rejects duplicates. Hashing, TLS, and password rotation do not prevent application-layer replay.

Question 2: Option C — Pass-the-hash
Pass-the-hash occurs when an attacker captures an NTLM password hash and uses that hash as the authenticator, without cracking it to plaintext. Session hijacking uses a stolen session token or cookie, Kerberoasting targets Kerberos service account tickets, and replay involves retransmitting a captured authentication message.

Question 3: Option C — HttpOnly
HttpOnly prevents client-side scripts from reading the cookie, which blocks theft through XSS. Secure ensures the cookie is sent only over HTTPS, SameSite limits cross-site sending, and Domain defines where the cookie is valid; none of those directly stop JavaScript access.

Question 4: Option A — Set session cookies with the Secure flag and enforce HTTPS with HSTS.
The core problem is that the session cookie is transmitted without encryption and can be captured on-path. Secure cookies are sent only over HTTPS, and HSTS enforces HTTPS connections, preventing cleartext exposure. Client-side storage, shorter timeouts, and compression/caching do not prevent on-path cookie theft.

Question 5: Option B — A timestamped authenticator
Kerberos uses authenticators that include a timestamp and are encrypted with a session key. The server checks the timestamp to reject old or replayed authenticators. NTLM hash caching is related to Pass-the-Hash, cookies are web session tokens, and permanent service tickets would increase replay risk rather than prevent it.

Question 6: Option B — Deploy Credential Guard and restrict privileged accounts to dedicated administration systems.
Credential Guard helps isolate and protect NTLM password hashes and other secrets from theft, reducing Pass-the-Hash risk. Restricting privileged accounts to dedicated admin systems limits lateral movement. Browser cookie flags, CAPTCHA, and Kerberos encryption for service accounts do not directly protect cached workstation credential hashes.

Question 7: Option C — Use a server-generated random challenge in each authentication attempt and include it in the response.
A server-generated random challenge, or nonce, makes each authentication exchange unique. Reusing a captured response will not match the new challenge. Longer passwords, Base64 encoding, and logging responses do not prevent replay of the captured authentication value.

Question 8: Option C — Regenerate the session ID immediately after successful authentication.
This describes session fixation: the attacker fixes a known session ID before login and reuses it afterward. Regenerating the session ID after authentication invalidates the attacker's known ID. HttpOnly helps with XSS theft but not fixation, login nonces address replay, and NTLM is unrelated to web session management.

Question 9: Option B — Pass-the-hash using a stolen credential hash
Successful NTLM authentications to many servers without interactive password entry are a common Pass-the-Hash indicator, especially when a tool uses a stolen hash as the credential. XSS cookie theft, Kerberos ticket replay, and session fixation would involve web cookies or Kerberos artifacts, not this NTLM authentication pattern.

Question 10: Option C — Track each passcode as used and bind it to the specific authentication session or challenge.
Intercepting and reusing a valid OTP within its validity window is a replay attack. Marking OTPs as used and binding them to a session or challenge prevents the same code from being accepted twice. Longer codes, SMS delivery, and client-side storage do not stop replay and may weaken security.